

Maven 完整使用手册（标签+命令+插件+SpringBoot专项配置）



本文主要介绍 maven 常用配置标签作用，常用打包插件 以及resource的占位解析

一、Maven 核心 POM 标签详解

代码块

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!-- Maven POM 核心配置文件，所有项目构建、依赖、插件、资源配置的核心入口 -->
3  <project xmlns="http://maven.apache.org/POM/4.0.0"
4          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
6                               http://maven.apache.org/xsd/maven-4.0.0.xsd">
7      <!-- ===== 1.项目基础核心版本（固定必填）
8      ===== -->
9      <modelVersion>4.0.0</modelVersion>
10
11     <!-- ===== 2.项目基础坐标（项目唯一标识，必填）
12     ===== -->
13     <!-- groupId: 组织/公司反向域名，用于区分不同项目组、公司项目 -->
14     <groupId>com.company</groupId>
15     <!-- artifactId: 当前模块唯一名称，Maven仓库标识，模块名唯一 -->
16     <artifactId>maven-core-demo</artifactId>
17     <!-- version: 项目版本号，分为快照SNAPSHOT(开发版)、正式RELEASE(生产版) -->
18     <version>2.7.15-SNAPSHOT</version>
19     <!-- packaging: 项目打包类型
20     jar: 普通Java/SpringBoot项目，默认值
21     war: 传统web项目，外置tomcat部署
22     pom: 父工程/BOM工程，仅管理依赖版本、插件，无编译产物，可全局传递公共依赖
23     -->
24     <packaging>jar</packaging>
25     <!-- name: 项目展示名称，可选，用于展示识别 -->
26     <name>Maven核心知识点Demo项目</name>
```

```

25 <!-- description: 项目功能描述, 可选 -->
26 <description>整合所有Maven核心POM标签、依赖、资源、插件配置示例</description>
27
28 <!-- ===== 3.parent父工程标签 (版本统一继承)
===== -->
29 <!-- 作用: 继承父工程所有依赖版本、插件配置、全局属性, 无需手动管理Spring全家桶版本
-->
30 <parent>
31     <groupId>org.springframework.boot</groupId>
32     <artifactId>spring-boot-starter-parent</artifactId>
33     <version>2.7.15</version>
34     <relativePath/> <!-- 不读取本地pom, 强制从Maven仓库拉取父POM -->
35 </parent>
36
37 <!-- ===== 4.properties全局属性标签 (全局占位符)
===== -->
38 <!-- 作用: 全局定义变量, 整个POM文件、资源配置文件yml/properties均可通过${变量名}
引用
    统一版本、编码、JDK版本, 便于后期统一维护修改
-->
39
40
41 <properties>
42     <!-- JDK编译版本 -->
43     <java.version>1.8</java.version>
44     <maven.compiler.source>1.8</maven.compiler.source>
45     <maven.compiler.target>1.8</maven.compiler.target>
46     <!-- 项目统一编码 -->
47     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
48     <!-- 自定义依赖版本全局变量 -->
49     <mysql.version>8.0.30</mysql.version>
50 </properties>
51
52 <!-- ===== 5.dependencyManagement 依赖版本锁定 (只锁版本, 不引
入依赖) ===== -->
53 <!-- 核心特性:
54     1、仅统一锁定依赖版本, 子模块/当前项目引入依赖无需写version
55     2、不会自动引入依赖, 需要手动dependencies引入
56     3、多用于父工程统一管理中间件、框架版本, 解决版本混乱问题
57 -->
58 <dependencyManagement>
59     <dependencies>
60         <dependency>
61             <groupId>mysql</groupId>
62             <artifactId>mysql-connector-java</artifactId>
63             <version>${mysql.version}</version>
64         </dependency>
65     </dependencies>
66 </dependencyManagement>

```

```

67
68      <!-- ===== 6.dependencies 项目依赖（自动传递依赖）
===== -->
69      <!-- 所有项目运行所需依赖均在此引入，支持依赖传递、依赖排除、作用域控制 -->
70      <dependencies>
71          <!-- SpringBoot Web核心依赖 -->
72          <dependency>
73              <groupId>org.springframework.boot</groupId>
74              <artifactId>spring-boot-starter-web</artifactId>
75              <!-- exclusions: 排除依赖，解决jar包冲突、日志冲突、多余依赖问题 -->
76              <exclusions>
77                  <exclusion>
78                      <groupId>org.slf4j</groupId>
79                      <artifactId>slf4j-log4j12</artifactId>
80                  </exclusion>
81              </exclusions>
82          </dependency>
83
84      <!-- ===== Maven六大scope作用域完整示例
===== -->
85      <!-- 1.compile【默认】：编译、测试、运行全部生效，支持依赖传递 -->
86      <!-- 2.test：仅单元测试阶段生效，不打包、不传递（如junit） -->
87      <!-- 3.provided：编译需要，运行时由容器/环境提供，不打入最终包、不传递 -->
88      <!-- 4.runtime：运行、测试生效，编译阶段不需要（如mysql驱动） -->
89
90
91      <!-- 5.system【特殊作用域】：本地系统依赖，不走Maven仓库
92      特性：需配置systemPath绝对路径、编译运行生效、支持传递、可打入包
93      缺点：可移植性极差，团队/流水线不兼容，企业禁止常规使用，仅适配无私服闭源
94      Jar
95      -->
96      <dependency>
97          <groupId>axis</groupId>
98          <artifactId>axis</artifactId>
99          <version>1.4.17</version>
100          <scope>system</scope>
101          <systemPath>D:\lib\axis.jar</systemPath>
102      </dependency>
103
104      <!-- 6.import：仅用于BOM版本管控，导入POM统一版本，不引入、不传递依赖 -->
105      </dependencies>
106
107      <!-- ===== 7.BOM版本统一导入（import作用域专属）
===== -->
108      <!-- 不通过继承SpringBoot父工程时，通过import导入BOM统一版本，无依赖传递 -->
109      <!-- 统一第三方组件版本，不继承父工程也能管控版本 -->
110      <dependencyManagement>

```

```

110     <dependencies>
111         <dependency>
112             <groupId>org.springframework.cloud</groupId>
113             <artifactId>spring-cloud-dependencies</artifactId>
114             <version>2021.0.5</version>
115             <type>pom</type>
116             <scope>import</scope>
117         </dependency>
118     </dependencies>
119 </dependencyManagement>
120
121 <!-- ===== 8.build构建核心配置（资源+插件）
===== -->
122 <build>
123     <!-- 打包文件名 -->
124     <finalName>${project.artifactId}-${profiles.env}</finalName>
125     <!-- 8.1 resources资源文件配置
126         filtering开关说明：
127         1、filtering=true：开启占位符替换，yml/properties可读取pom全局变量
128         2、filtering=false：原样输出，MyBatis Mapper XML必须关闭，防止#{ }被
Maven误解析报错
129     -->
130     <resources>
131         <!-- 控制编译时复制哪些配置文件、开启占位符替换 -->
132
133         <resource>
134             <directory>src/main/resources</directory>
135             <!-- 业务配置文件：开启占位符替换 -->
136             <!-- 开启占位符 ${xxx} 替换 -->
137             <filtering>true</filtering>
138             <includes>
139                 <include>**/*.yml</include>
140                 <include>**/*.properties</include>
141             </includes>
142         </resource>
143         <!-- MyBatis映射文件：关闭占位符过滤，防止SQL语法报错 -->
144         <resource>
145             <directory>src/main/resources</directory>
146             <!-- 关闭占位符替换 -->
147             <!-- 原样输出资源文件内容，不替换任何占位符，MyBatis Mapper XML 文件必
须关闭，避免 SQL 中 #{ } 被Maven误解析替换，导致SQL报错 -->
148             <filtering>false</filtering>
149             <includes>
150                 <include>mapper/**/*.xml</include>
151             </includes>
152         </resource>
153     </resources>

```

```

154
155 <!-- 8.2 核心插件配置 构建生命周期执行的工具，编译、打包、测试都靠插件-->
156 <plugins>
157     <!-- 编译插件：统一JDK编译版本、编码 -->
158     <plugin>
159         <groupId>org.apache.maven.plugins</groupId>
160         <artifactId>maven-compiler-plugin</artifactId>
161         <version>3.8.1</version>
162         <configuration>
163             <source>${java.version}</source>
164             <target>${java.version}</target>
165             <encoding>UTF-8</encoding>
166         </configuration>
167     </plugin>
168
169     <!-- 资源插件【重点】：占位符分隔符、编码、默认占位符配置
170         重要知识点: delimiters、useDefaultDelimiters 属于插件配置，不属于
resources标签
171         支持双占位符：${*}、@变量@，解决语法冲突
172         -->
173     <plugin>
174         <groupId>org.apache.maven.plugins</groupId>
175         <artifactId>maven-resources-plugin</artifactId>
176         <version>3.3.1</version>
177         <configuration>
178             <encoding>UTF-8</encoding>
179             <!-- 自定义双占位符分隔符 -->
180             <delimiters>
181                 <delimiter>${*}</delimiter> <!-- 传统${xxx}占位符 -->
182                 <delimiter>@</delimiter> <!-- 安全@xxx@占位符，规避语法
冲突 -->
183             </delimiters>
184             <!-- 保留Maven默认内置变量，防止系统变量失效 -->
185             <useDefaultDelimiters>true</useDefaultDelimiters>
186         </configuration>
187     </plugin>
188
189 </plugins>
190 </build>
191
192 <!-- ===== 9.多环境Profile配置 (dev/prod/test)
===== -->
193 <profiles>
194     <profile>
195         <id>dev</id>
196         <properties>
197             <spring.profiles.active>dev</spring.profiles.active>

```

```

198         </properties>
199         <activation>
200             <activeByDefault>true</activeByDefault> <!-- 默认激活开发环境 -->
201         </activation>
202     </profile>
203     <profile>
204         <id>prod</id>
205         <properties>
206             <spring.profiles.active>prod</spring.profiles.active>
207         </properties>
208     </profile>
209 </profiles>
210
211 <!-- ===== 10. 仓库配置 (依赖/插件仓库)
===== -->
212 <!-- 依赖仓库：用于下载项目jar依赖 -->
213 <repositories>
214     <repository>
215         <id>aliyun-central</id>
216         <url>https://maven.aliyun.com/repository/public</url>
217     </repository>
218 </repositories>
219
220 <!-- 插件仓库：用于下载maven编译、打包插件 -->
221 <pluginRepositories>
222     <pluginRepository>
223         <id>aliyun-plugin</id>
224         <url>https://maven.aliyun.com/repository/public</url>
225     </pluginRepository>
226 </pluginRepositories>
227
228 </project>

```

二、Maven 常用命令（生命周期 + 参数）

Maven 三套生命周期：clean、default、site，执行后续阶段会自动执行前置阶段。

1. Clean 清理生命周期

代码块

```
1 mvn clean # 删除target编译输出目录，清除旧打包文件
```

2. Default 构建生命周期（高频）

代码块

```
1 mvn compile # 仅编译java代码, 生成class到target
2 mvn test # 编译+执行单元测试
3 mvn package # 编译+测试+打包(jar/war), 输出target下
4 mvn install # package + 安装到本地maven仓库, 本地其他模块可依赖
5 mvn deploy # install + 上传到远程私服仓库 (公司仓库)
6 mvn validate # 校验项目结构、依赖完整性
7 mvn verify # 执行集成测试, 校验包完整性
```

3. 多模块命令

代码块

```
1 mvn clean install -Dmaven.test.skip=true # 跳过测试打包安装
2 mvn clean package -pl module-demo -am # 只打包指定模块, 同时打包依赖模块
3 mvn clean package -pl module-demo # 仅打包当前模块, 不构建依赖
```

4. 常用附加参数

代码块

```
1 -Dmaven.test.skip=true # 跳过单元测试 (速度快)
2 -Pprod # 激活prod环境profile配置
3 -X # 打印debug日志, 排查依赖冲突
4 -U # 强制更新快照SNAPSHOT依赖
```

5. Profile 环境切换（开发 / 测试 / 生产）

pom 中定义多环境，命令 `-P` 切换

代码块

```
1 <profiles>
2   <profile>
3     <id>dev</id>
4     <properties>
5       <env.active>dev</env.active>
6     </properties>
7   </profile>
8 </profiles>
```

启动命令： `mvn package -Pdev`

三、Maven 核心常用插件及作用

插件配置统一放在 `<build><plugins><plugin>` 内

1. maven-compiler-plugin 编译插件（必备）

作用：指定 JDK 版本编译 Java 代码

代码块

```
1  <plugin>
2    <groupId>org.apache.maven.plugins</groupId>
3    <artifactId>maven-compiler-plugin</artifactId>
4    <version>3.8.1</version>
5    <configuration>
6      <source>${java.version}</source>
7      <target>${java.version}</target>
8      <encoding>UTF-8</encoding>
9    </configuration>
10 </plugin>
```

2. maven-resources-plugin 资源文件插件

配合 build/resources 标签实现资源拷贝、目录过滤、占位符替换，是SpringBoot资源处理核心插件。占位符分隔符、默认占位符开关、编码配置均在此插件中定义（标准规范写法），企业级统一自定义占位符配置如下：

标准规范：maven-resources-plugin 自定义占位符分隔符配置

```
1  <plugin>
2    <groupId>org.apache.maven.plugins</groupId>
3    <artifactId>maven-resources-plugin</artifactId>
4    <version>3.3.1</version>
5    <configuration>
6      <!-- 统一资源文件编码 -->
7      <encoding>UTF-8</encoding>
8      <!-- 自定义多套占位符分隔符（核心配置） -->
9      <delimiters>
10        <delimiter>${*}</delimiter> <!-- 经典默认格式 ${xxx} -->
11        <delimiter>@</delimiter> <!-- 安全格式 @xxx@，规避语法冲突 -->
12      </delimiters>
13      <!-- 是否保留Maven默认占位符规则，开启可兼容系统内置变量 -->
14      <useDefaultDelimiters>true</useDefaultDelimiters>
15    </configuration>
16 </plugin>
```

核心配置详解 & 实战场景

- **delimiters 多占位符兼容**：同时支持 `${变量}`、`@变量@` 两种写法，适配新旧项目、多团队开发规范，解决 `${}` 与SpringEL、MyBatis语法冲突问题，`@xxx@` 是SpringBoot推荐安全占位符。
- **useDefaultDelimiters 作用**：默认true，保留Maven原生内置变量（`${project.artifactId}`、`${project.version}` 等）；若设置为false，仅生效自定义分隔符，原生 `${}` 内置变量失效。
- **配置区分（必记）**：`filtering` 占位符总开关、资源目录配置写在 `build/resources`；`delimiters` 分隔符规则、编码、默认开关写在 `maven-resources-plugin` 插件内，是标准企业写法。

完整联动使用示例

1、pom全局变量定义

代码块

```
1 <properties>
2   <jdbc.url>jdbc:mysql://127.0.0.1:3306/test</jdbc.url>
3   <jdbc.driver>com.mysql.cj.jdbc.Driver</jdbc.driver>
4 </properties>
```

2、build开启资源过滤总开关

代码块

```
1 <build>
2   <resources>
3     <resource>
4       <directory>src/main/resources</directory>
5       <filtering>true</filtering>
6       <includes>
7         <include>**/*.yml</include>
8         <include>**/*.properties</include>
9       </includes>
10    </resource>
11    <!-- MyBatis xml关闭过滤，防止${}被误解析 -->
12    <resource>
13      <directory>src/main/resources</directory>
14      <filtering>false</filtering>
15      <includes>
16        <include>mapper/**/*.xml</include>
17      </includes>
18    </resource>
19  </resources>
20 </build>
```

3、yml配置双占位符混用（均可正常替换）

代码块

```
1  spring:
2    datasource:
3      url: ${jdbc.url}
4      driver-class-name: @jdbc.driver@
```

3. yaml-properties-maven-plugin Yaml 解析插件（反向读取yml配置）

核心作用：常规开发是 yml 读取 pom 变量，本插件实现**反向取值**：解析 application.yml 配置文件，将 yml 中的参数转为 Maven 全局属性，可在 pom.xml 中通过 `${key}` 直接引用。

常用场景：读取 yml 配置的服务端口、环境标识、服务名称，动态自定义打包后的 Jar 包文件名，方便多环境部署区分管理。

参考文档：https://blog.csdn.net/weixin_44338092/article/details/132186110

yaml-properties-maven-plugin 完整配置

```
1  <plugin>
2    <groupId>it.ozimov</groupId>
3    <artifactId>yaml-properties-maven-plugin</artifactId>
4    <version>1.1.3</version>
5    <executions>
6      <execution>
7        <!-- initialize生命周期最前置阶段，优先解析yml，打包时才能正常读取变量 -->
8        <phase>initialize</phase>
9        <goals>
10         <!-- 核心目标：读取yml配置并注入Maven上下文为全局属性 -->
11         <goal>read-project-properties</goal>
12       </goals>
13       <configuration>
14         <!-- 指定需要解析的yml配置文件，多文件可追加多个<file>标签 -->
15         <files>
16           <file>src/main/resources/application.yml</file>
17         </files>
18       </configuration>
19     </execution>
20   </executions>
21 </plugin>
```

配套 build 全局配置（动态 jar 包名 + 资源过滤）

代码块

```

1  <build>
2      <!--suppress UnresolvedMavenProperty 屏蔽IDEA变量未识别警告 -->
3      <!-- 动态拼接打包后的jar名称, 直接读取yml内server.port端口参数 -->
4      <finalName>${project.name}${server.port}</finalName>
5
6      <resources>
7          <resource>
8              <directory>src/main/resources</directory>
9              <!-- 反向读取yml场景关闭过滤, 避免maven篡改yml原始内容 -->
10             <filtering>>false</filtering>
11         </resource>
12         <!-- mapper xml单独配置, 关闭过滤防止SQL中#{ }被误解解析报错 -->
13         <resource>
14             <directory>src/main/resources</directory>
15             <filtering>>false</filtering>
16             <includes>
17                 <include>mapper/**/*.xml</include>
18             </includes>
19         </resource>
20     </resources>
21 </build>

```

配套 application.yml 示例配置

代码块

```

1  server:
2      port: 8080
3  spring:
4      application:
5          name: demo-service

```

使用效果与核心注意事项

- **打包命令：**执行 `mvn clean package`，自动读取yml端口拼接包名，示例包名：`Maven核心知识点Demo项目8080.jar`
- **执行阶段必配：**phase 必须为 initialize（Maven最前置生命周期），保证优先解析yml，后续打包流程可正常读取变量
- **过滤配置要求：**反向读取yml场景必须设置 `filtering=false`，禁止Maven修改、篡改yml原始配置内容
- **文件格式限制：**该插件仅支持 yml/yaml 文件解析，无法读取 properties 格式配置文件
- **IDEA警告处理：**pom中引用yml变量会出现未识别警告，添加注释 `<!--suppress UnresolvedMavenProperty -->` 即可消除

4. maven-test-plugin 单元测试插件

执行 JUnit/TestNG 测试，跳过测试： `-Dmaven.test.skip=true`

4. maven-clean-plugin 清理插件

对应 `mvn clean`，删除 target 目录，无需手动配置，内置默认执行。

5. maven-jar-plugin jar 打包插件

普通 Java 项目打包成标准 jar；SpringBoot 一般不用，由 spring-boot-maven-plugin 替代。

6. spring-boot-maven-plugin (SpringBoot 核心插件)

重中之重，SpringBoot 专属打包插件

作用：

1. 打包可执行 FatJar（内嵌 Tomcat，java -jar 直接运行）
2. 提供启动、停止、构建镜像能力
3. 分离依赖包、配置文件（外部化配置）

完整配置：

代码块

```
1  <plugin>
2      <groupId>org.springframework.boot</groupId>
3      <artifactId>spring-boot-maven-plugin</artifactId>
4      <version>2.7.15</version>
5      <executions>
6          <execution>
7              <goals>
8                  <goal>repackage</goal> <!-- 重新打包为可执行jar -->
9              </goals>
10         </execution>
11     </executions>
12     <configuration>
13         <mainClass>com.company.demo.DemoApplication</mainClass> <!-- 启动类 -->
14         <excludes>
15             <!-- 打包时排除配置文件，外置读取 -->
16             <exclude>
17                 <groupId>*</groupId>
18                 <artifactId>*</artifactId>
19             </exclude>
20         </excludes>
21     </configuration>
22 </plugin>
23
24 <!-- 带 classifier=exec 企业常用完整配置 -->
```

```

25 <plugin>
26     <groupId>org.springframework.boot</groupId>
27     <artifactId>spring-boot-maven-plugin</artifactId>
28     <version>2.7.15</version>
29     <executions>
30         <execution>
31             <goals>
32                 <goal>repackage</goal>
33             </goals>
34         </execution>
35     </executions>
36     <configuration>
37         <!-- 可执行包标识, exec=executable可执行 -->
38         <classifier>exec</classifier>
39         <!-- 指定项目唯一启动入口 -->
40         <mainClass>com.inspur.TYZXApplication</mainClass>
41     </configuration>
42 </plugin>

```

classifier=exec 核心详解（企业高频用法）

classifier是Maven构件后缀标识，**仅用于SpringBoot多模块项目**，核心作用是隔离包类型，解决模块「可被依赖」和「可独立部署」的冲突。

即打两个包 一个是依赖包，一个是运行包（可以java -jar 直接运行），默认不配置参数只打包运行包。

代码块

```
1 mvn spring-boot:run # 直接本地启动SpringBoot项目，无需打包
```

7. maven-war-plugin 打 war 包插件

外置 Tomcat 部署使用，排除内嵌 tomcat 依赖。

8. maven-source-plugin 源码打包插件

上传源码到仓库，开发时可查看源码

代码块

```

1 <plugin>
2     <groupId>org.apache.maven.plugins</groupId>
3     <artifactId>maven-source-plugin</artifactId>
4     <executions>
5         <execution>

```

```

6         <goals>
7             <goal>jar-no-fork</goal>
8         </goals>
9     </execution>
10 </executions>
11 </plugin>

```

9. maven-dependency-plugin 依赖分析插件

常用功能：拷贝依赖、分析冲突、列出依赖树

代码块

```

1 mvn dependency:tree # 打印完整依赖树，排查jar包冲突
2 mvn dependency:analyze # 检测未使用/缺失依赖

```

10. mybatis-generator-plugin MyBatis 代码生成插件

自动生成 Entity、Mapper 接口、XML 映射文件，CRUD 基础代码。

四、SpringBoot 项目专属 Maven 配置汇总

1. 资源文件占位符完整配置（解决 yml 读取 pom 变量）

场景：将 pom 中的版本、环境、项目名称注入 application.yml

pom.xml resources 配置：

代码块

```

1 <build>
2     <!-- 开启资源过滤，替换${}占位符 -->
3     <resources>
4         <resource>
5             <directory>src/main/resources</directory>
6             <filtering>true</filtering>
7             <includes>
8                 <include>**/*.yml</include>
9                 <include>**/*.properties</include>
10            </includes>
11        </resource>
12        <!-- mapper xml关闭过滤，防止#{}被解析出错 -->
13        <resource>
14            <directory>src/main/resources</directory>
15            <filtering>false</filtering>
16            <includes>
17                <include>mapper/**/*.xml</include>

```

```
18         </includes>
19     </resource>
20 </resources>
21 </build>
```

application.yml 使用占位符：

代码块

```
1  spring:
2    application:
3      name: ${project.artifactId} # 读取pom artifactId
4    datasource:
5      driver-class-name: com.mysql.cj.jdbc.Driver
6      version: ${mysql.version} # 读取pom中properties的mysql.version
```

内置 Maven 内置变量可直接使用：

- `${project.version}` 项目版本
- `${project.groupId}` 组织 ID
- `${project.artifactId}` 模块名
- `${project.build.sourceEncoding}` 编码

2. 外置配置文件打包分离（生产常用）

通过 spring-boot-maven-plugin 将配置文件移出 jar 包，部署时外部修改 yml 无需重新打包

代码块

```
1  <plugin>
2    <groupId>org.springframework.boot</groupId>
3    <artifactId>spring-boot-maven-plugin</artifactId>
4    <configuration>
5      <layout>ZIP</layout>
6      <!-- 将config目录下配置输出到包外 -->
7      <requiresUnpack>
8        <dependency>
9          <groupId>${project.groupId}</groupId>
10         <artifactId>${project.artifactId}</artifactId>
11       </dependency>
12     </requiresUnpack>
13   </configuration>
14 </plugin>
```

3. 统一依赖管理（两种方案）

方案 1：继承 spring-boot-starter-parent（最简）

代码块

```
1  <parent>
2      <groupId>org.springframework.boot</groupId>
3      <artifactId>spring-boot-starter-parent</artifactId>
4      <version>2.7.15</version>
5      <relativePath/>
6  </parent>
```

引入 starter 无需写 version。

方案 2：import 导入 BOM（不继承父工程）

代码块

```
1  <dependencyManagement>
2      <dependencies>
3          <dependency>
4              <groupId>org.springframework.boot</groupId>
5              <artifactId>spring-boot-dependencies</artifactId>
6              <version>2.7.15</version>
7              <type>pom</type>
8              <scope>import</scope>
9          </dependency>
10     </dependencies>
11 </dependencyManagement>
```

4. 跳过测试打包（部署流水线必备）

代码块

```
1  mvn clean package -Dmaven.test.skip=true
```

5. 内置 Tomcat 排除（打 war 外置 Tomcat）

代码块

```
1  <dependency>
2      <groupId>org.springframework.boot</groupId>
3      <artifactId>spring-boot-starter-web</artifactId>
```



```

4      <exclusions>
5          <exclusion>
6              <groupId>org.springframework.boot</groupId>
7              <artifactId>spring-boot-starter-tomcat</artifactId>
8          </exclusion>
9      </exclusions>
10 </dependency>
11 <!-- 提供servlet api -->
12 <dependency>
13     <groupId>javax.servlet</groupId>
14     <artifactId>javax.servlet-api</artifactId>
15     <scope>provided</scope>
16 </dependency>

```

6. 多环境 Profile 配置示例

pom:

代码块

```

1  <profiles>
2      <profile>
3          <id>dev</id>
4          <properties>
5              <spring.profiles.active>dev</spring.profiles.active>
6          </properties>
7          <activation>
8              <activeByDefault>true</activeByDefault> <!-- 默认激活dev -->
9          </activation>
10     </profile>
11     <profile>
12         <id>prod</id>
13         <properties>
14             <spring.profiles.active>prod</spring.profiles.active>
15         </properties>
16     </profile>
17 </profiles>

```

打包生产包: `mvn clean package -Pprod`

五、Maven 引入第三方本地Jar包（三种实战方案）

<https://www.cnblogs.com/fightingtong/p/14187884.html>

<https://blog.csdn.net/cyw456/article/details/130634967>

日常开发中会遇到 **Maven 私服/中央仓库不存在的第三方闭源Jar、老旧依赖Jar**，无法直接通过常规依赖引入。此处整理企业常用三种解决方案，涵盖 Jar 项目、War 项目专属配置，适配不同部署场景。

方案一：本地Jar安装至Maven仓库（标准规范、推荐）

核心思想：将本地第三方Jar手动安装到本地Maven仓库，后续通过常规依赖引入，完全遵循Maven规范，无环境兼容问题、可传递依赖、支持团队共享。

1. 前置准备：Maven环境配置

1. 下载Maven压缩包并解压至本地无中文、无空格路径，如：`D:\apache-maven-3.3.9`
2. 新建系统环境变量：`MAVEN_HOME`，变量值为Maven解压根目录
3. 在系统Path变量中追加：`%MAVEN_HOME%\bin;`
4. 修改Maven仓库路径（解决C盘占用问题）：打开 `MAVEN_HOME/conf/settings.xml`，修改本地仓库地址
`<localRepository>E:\Maven\home\repository</localRepository>`
5. 环境校验：CMD窗口执行 `mvn -version`，输出版本信息即安装成功（需提前配置JDK环境）

2. 执行命令安装本地Jar到仓库

通用安装命令：

代码块

```
1 mvn install:install-file -Dfile=本地Jar绝对路径 -DgroupId=自定义组ID -DartifactId=自定义项目ID -Dversion=版本号 -Dpackaging=jar
```

实战示例：安装axis、jaxrpc第三方Jar

代码块

```
1 mvn install:install-file -Dfile=D:\lib\axis.jar -DgroupId=axis -DartifactId=axis -Dversion=1.4.17 -Dpackaging=jar
2 mvn install:install-file -Dfile=D:\lib\jaxrpc.jar -DgroupId=axis -DartifactId=jaxrpc -Dversion=1.4.17 -Dpackaging=jar
```

执行成功后：本地仓库路径 `E:\Maven\home\repository` 下会自动生成对应依赖文件夹及Jar包。

3. POM引入常规依赖

安装完成后，和普通Maven依赖一致，直接引入即可，无需system作用域：

```

1 代码块<dependencies>
2      <dependency>
3          <groupId>axis</groupId>
4          <artifactId>axis</artifactId>
5          <version>1.4.17</version>
6      </dependency>
7      <dependency>
8          <groupId>axis</groupId>
9          <artifactId>jaxrpc</artifactId>
10         <version>1.4.17</version>
11     </dependency>
12 </dependencies>

```

方案优缺点

-  优点：标准Maven规范、支持依赖传递、适配CI/CD流水线、团队协作无冲突、可上传私服共享
-  缺点：首次使用需要手动执行安装命令

方案二：system作用域+打包插件（临时快速方案、项目常用）

核心思想：不安装Jar到Maven仓库，直接通过 `system` 作用域指定本地Jar绝对路径，配合SpringBoot打包插件配置，将本地Jar打入最终可执行包。

1. SpringBoot Jar项目专属配置

默认情况下，system作用域的Jar不会被SpringBoot打包进FatJar，必须在 `spring-boot-maven-plugin` 中开启打包支持：

```

1 代码块
2 1 <build>
3   2 <plugins>
4     3 <plugin>
5       4 <groupId>org.springframework.boot</groupId>
6       5 <artifactId>spring-boot-maven-plugin</artifactId>
7       6 <configuration>
8         7 <!-- 核心配置：将system作用域的本地Jar打入最终可执行Jar包 -->
9         8 <includeSystemScope>true</includeSystemScope>
10      9 </configuration>
11    10 </plugin>
12  11 </plugins>
13 12 </build>

```

2. 配置system本地依赖

```

1  <dependency>
2      <groupId>axis</groupId>
3      <artifactId>axis</artifactId>
4      <version>1.4.17</version>
5      <scope>system</scope>
6      <systemPath>D:\lib\axis.jar</systemPath>
7  </dependency>
8
9  <dependency>
10     <groupId>axis</groupId>
11     <artifactId>jaxrpc</artifactId>
12     <version>1.4.17</version>
13     <scope>system</scope>
14     <systemPath>D:\lib\jaxrpc.jar</systemPath>
15 </dependency>

```

3. War项目专属适配配置

War包打包默认会将system依赖输出至 WEB-INF/lib-provided，Tomcat 仅识别 WEB-INF/lib 下的Jar，会导致项目启动报错，需配置 maven-war-plugin 手动拷贝Jar至正确目录：

代码块

```

1  <plugin>
2      <groupId>org.apache.maven.plugins</groupId>
3      <artifactId>maven-war-plugin</artifactId>
4      <configuration>
5          <webResources>
6              <resource>
7                  <!-- 项目根目录下的lib文件夹，存放所有第三方Jar -->
8                  <directory>lib</directory>
9                  <!-- 打包后输出到Tomcat可识别的WEB-INF/lib目录 -->
10                 <targetPath>WEB-INF/lib/</targetPath>
11                 <includes>
12                     <include>**/*.jar</include>
13                 </includes>
14             </resource>
15         </webResources>
16     </configuration>
17     另一种方式
18     <compilerArguments>
19         <extdirs>src/lib</extdirs>
20     </compilerArguments>
21     另一种方式

```

配置后打包效果：同时保留 `lib-provided` 文件夹，第三方Jar正常输出至 `WEB-INF/lib`，Tomcat可正常加载依赖。

方案优缺点

-  优点：无需安装Jar到仓库、配置简单、快速落地
-  缺点：依赖本地绝对路径、可移植性差、不推荐正式环境长期使用

方案三：手动放置 WEB-INF/lib（纯原生Web项目方案）

核心思想：无需任何Maven配置，纯手动方式引入第三方Jar，适配传统SSM、原生Web War项目。

操作步骤

1. 在项目 `WEB-INF` 目录下手动新建文件夹，命名为 `lib`
2. 将所有第三方本地Jar包直接复制粘贴至 `WEB-INF/lib` 目录下
3. IDE自动识别该目录Jar，编译、运行、打包时自动加载，无需在pom.xml配置任何依赖

方案优缺点

-  优点：零配置、简单粗暴、适配老旧Web项目
-  缺点：Maven无法管理该部分依赖、无版本管控、依赖无法传递、仅适合临时测试使用

三种方案选型总结（企业规范）

1. 正式项目、长期维护、流水线部署：优先选择 方案一（安装至Maven仓库）
2. 临时调试、快速开发、短期项目：选择 方案二（system作用域）
3. 老旧原生Web项目、零Maven依赖需求：选择 方案三（手动WEB-INF/lib）

六、补充关键总结

1. `packaging=pom` 模块专门用于统一管理依赖、版本，可全局传递公共依赖；
 2. `filtering` 是 SpringBoot 资源占位符核心开关，xml 映射文件必须关闭；
 3. `spring-boot-maven-plugin` 是 SpringBoot 独有插件，负责可执行 jar 打包与本地运行；
 4. `scope` 控制依赖打包与传递范围，冲突优先使用 `exclusion` 排除；
 5. Maven 命令遵循生命周期，执行 `package/install` 会自动执行 `compile`、`test` 前置步骤。
1. `packaging=pom` 模块专门用于统一管理依赖、版本，可全局传递公共依赖；
 2. `filtering` 是 SpringBoot 资源占位符核心开关，xml 映射文件必须关闭；
 3. `spring-boot-maven-plugin` 是 SpringBoot 独有插件，负责可执行 jar 打包与本地运行；

4. scope 控制依赖打包与传递范围，冲突优先使用 exclusion 排除；
5. Maven 命令遵循生命周期，执行 package/install 会自动执行 compile、test 前置步骤。

（注：文档部分内容可能由 AI 生成）